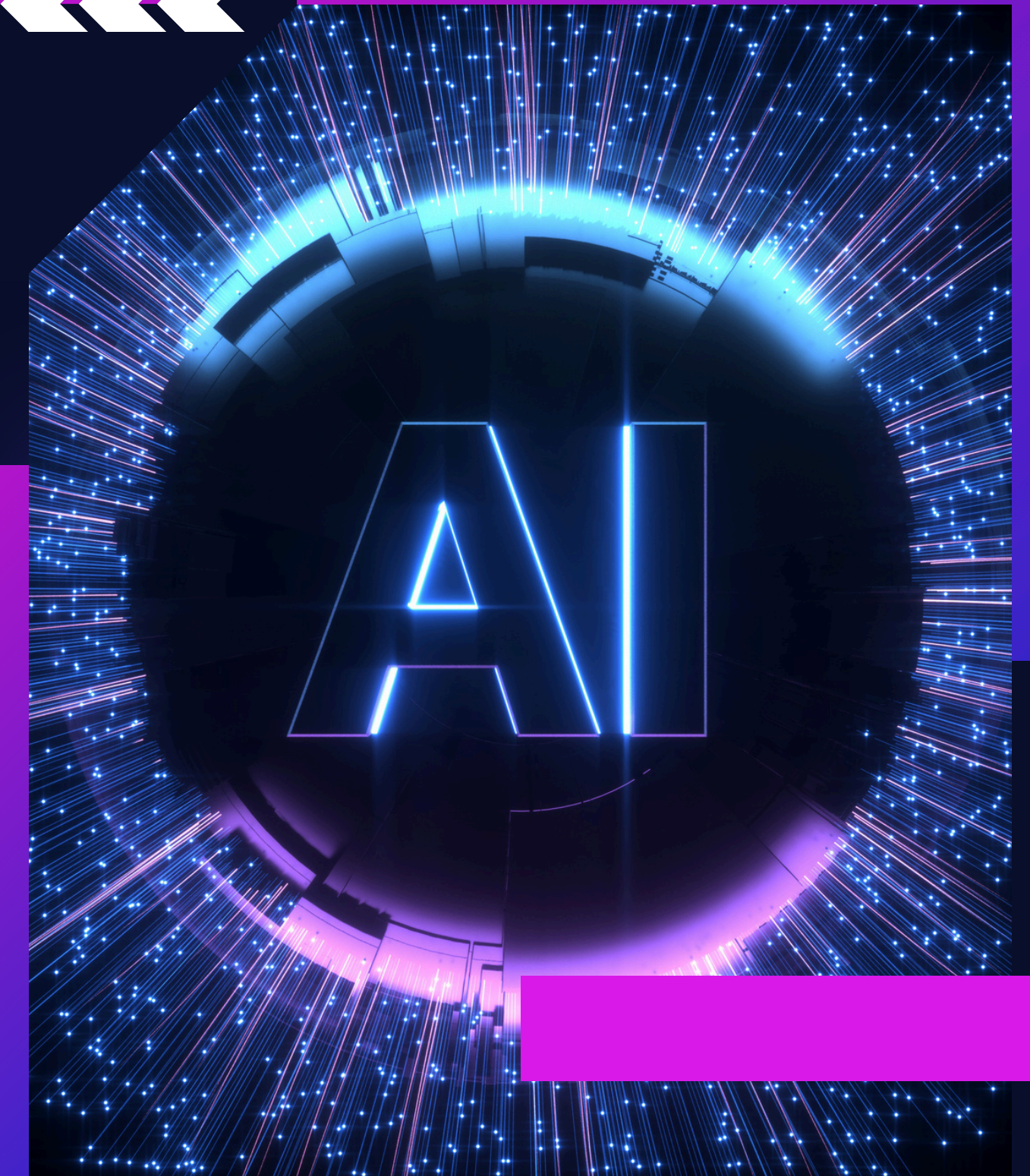


บทที่ 6 : การพัฒนาระบบแนะนำ

- 6.1 แนะนำระบบแนะนำ (Recommendation System)
- 6.2 การพัฒนา ML/AI Model สำหรับแนะนำสินค้า
- 6.3 การพัฒนา API สำหรับแนะนำสินค้า
- 6.4 การพัฒนา Web App สำหรับแนะนำสินค้า
- 6.5 การพัฒนา Mobile App สำหรับแนะนำสินค้า



6.1 แนะนำระบบแนะนำ (Recommendation System)

ระบบแนะนำ (Recommendation System) คือ ระบบที่ใช้เทคนิคทางคอมพิวเตอร์และสถิติเพื่อ วิเคราะห์ข้อมูลของผู้ใช้และพฤติกรรม ในการให้คำแนะนำที่ตรงกับความสนใจ ความต้องการ หรือพฤติกรรมของผู้ใช้นั้น ๆ

- **ตัวอย่างระบบแนะนำ**

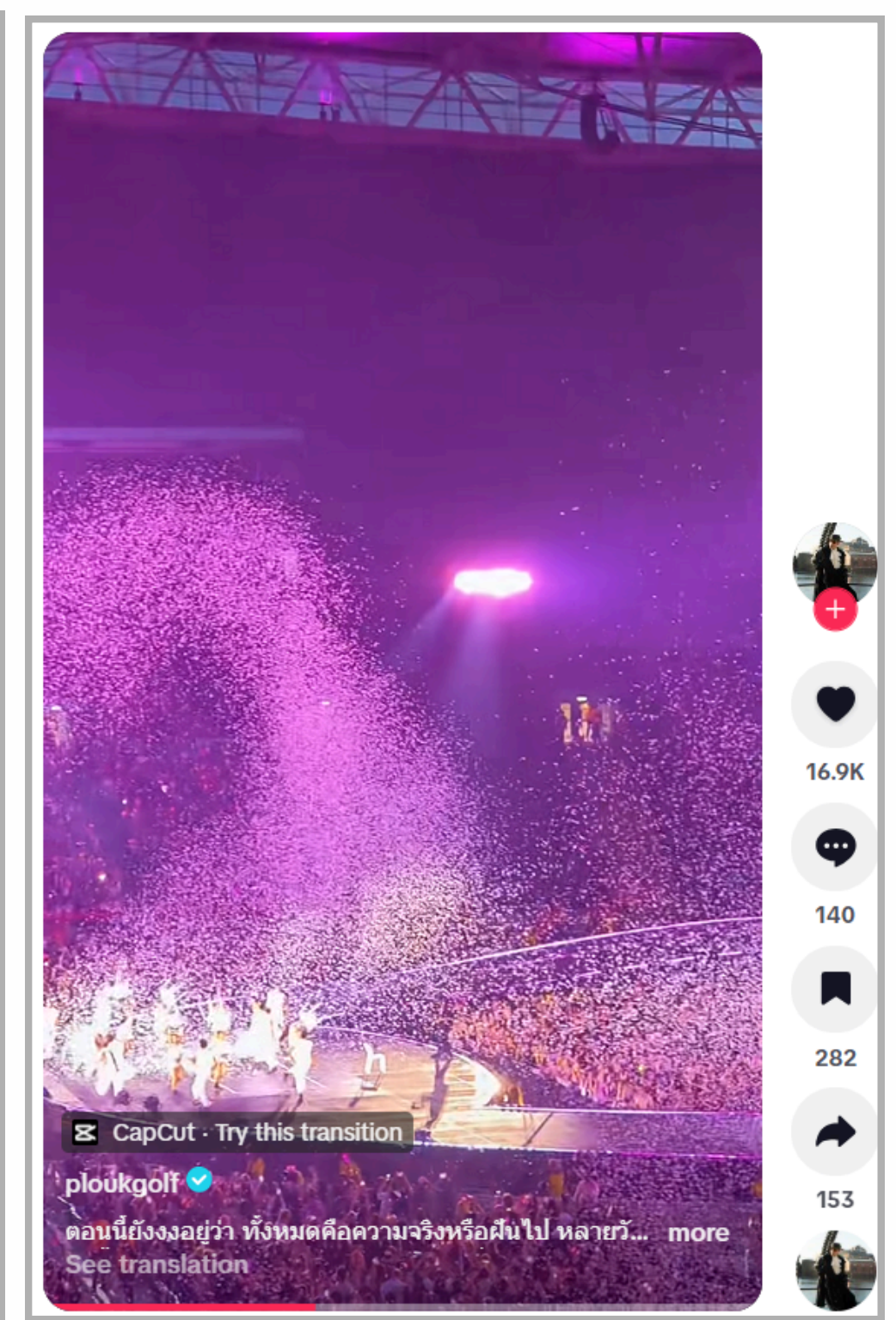
1. Facebook / Instagram / TikTok → แนะนำโพสต์หรือคอนเทนต์ที่ผู้ใช้น่าจะสนใจ
2. Netflix / YouTube → แนะนำหนังหรือวิดีโอที่น่าจะชอบ
3. Spotify → แนะนำเพลงใหม่ ๆ ที่ตรงกับสไตล์ของผู้ฟัง
4. Shopee / Lazada / Amazon → แนะนำสินค้าที่เกี่ยวข้องกับพฤติกรรมการซื้อ



Facebook



Instagram

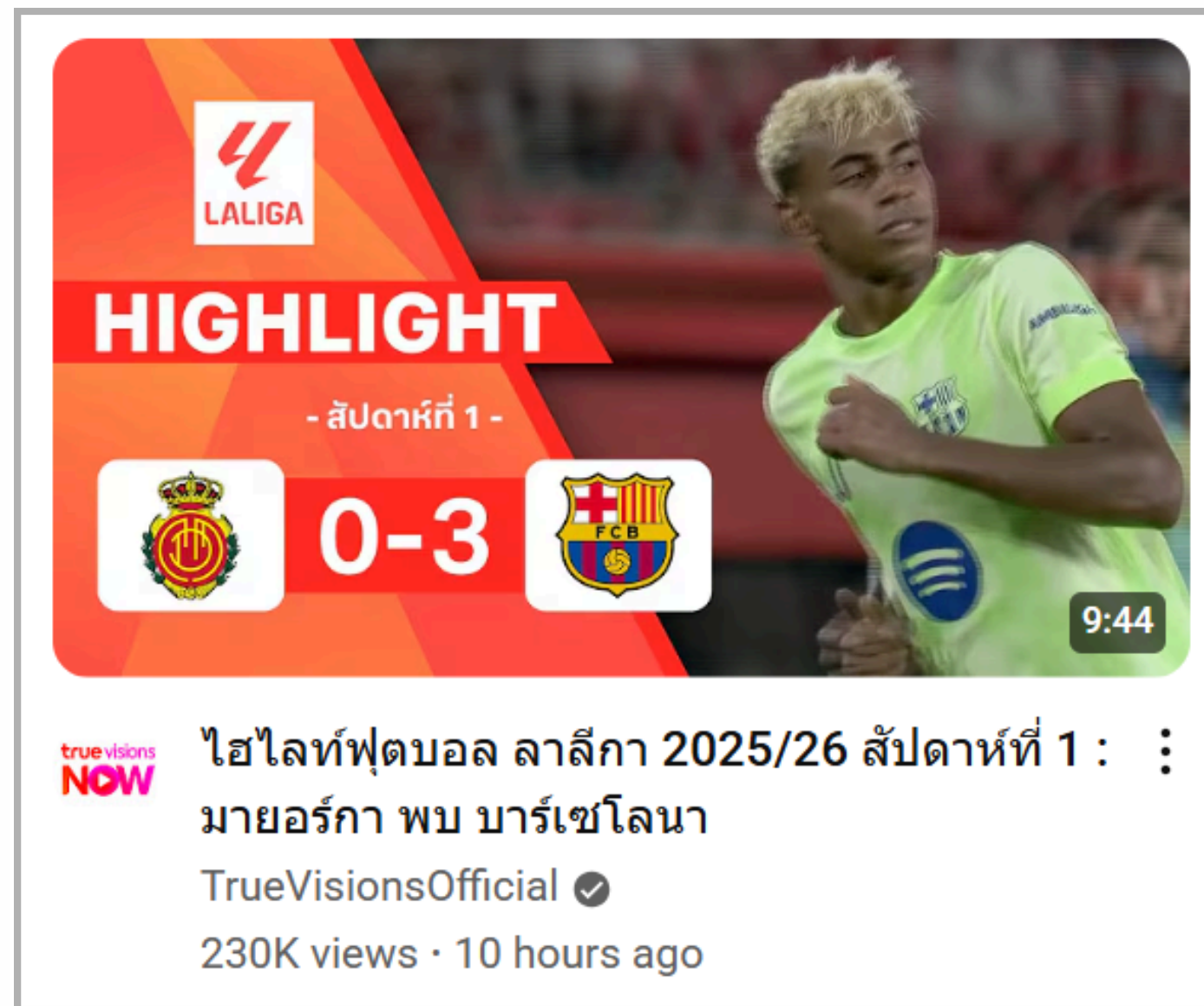


TikTok

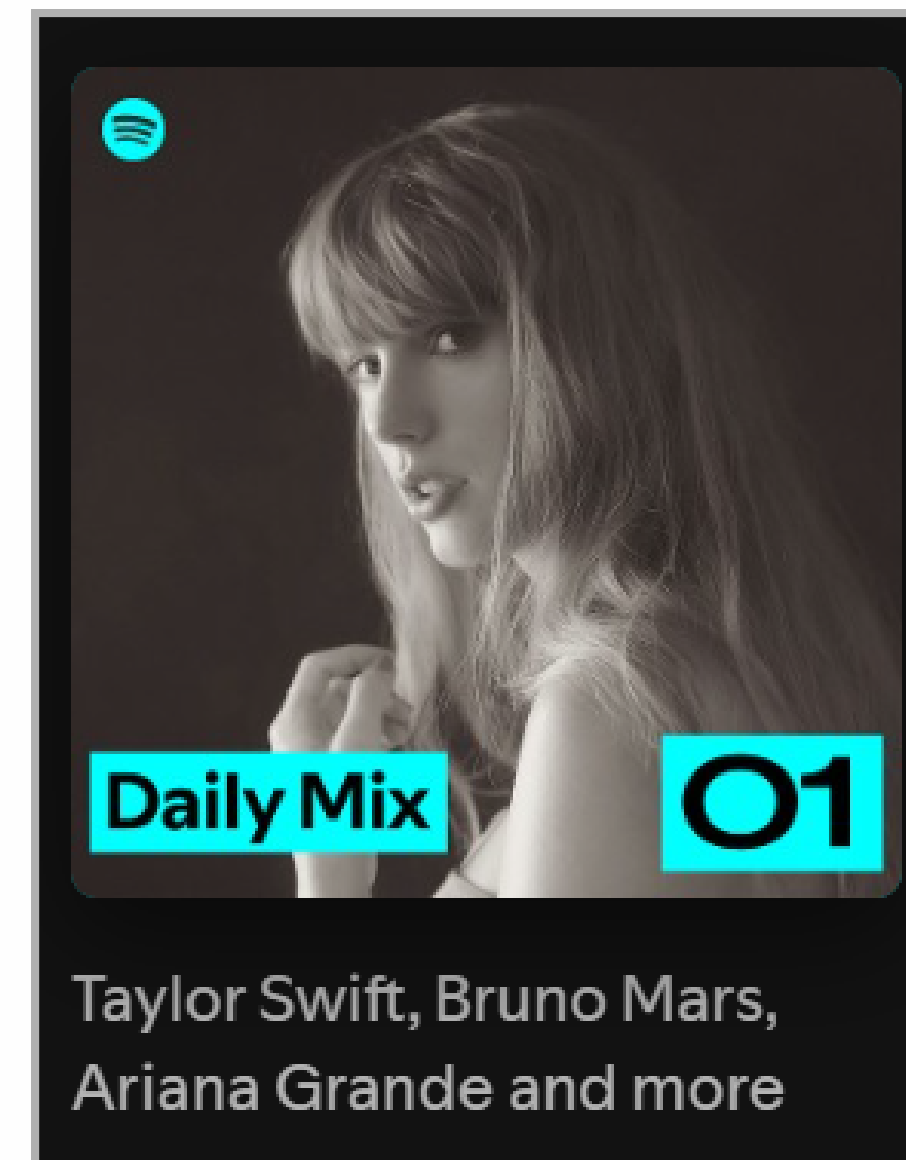
ภาพที่ 6-1 : ตัวอย่างระบบแนะนำ (Recommendation System)



Netflix

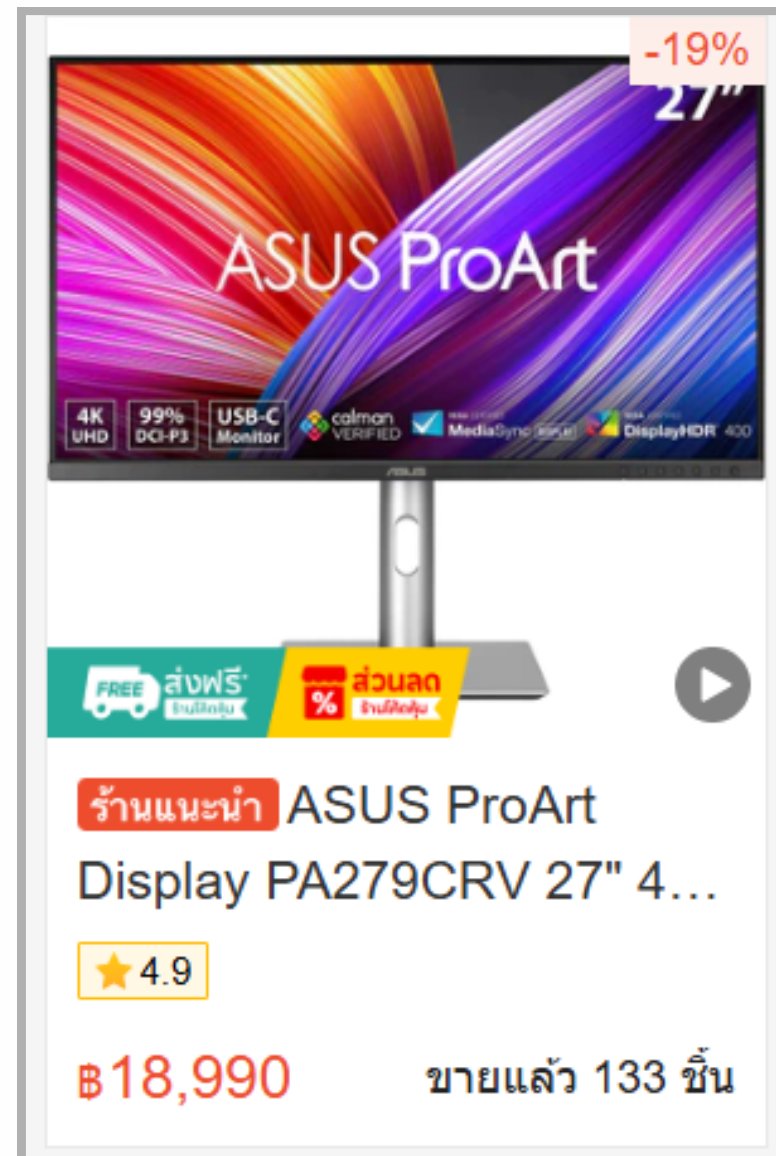


YouTube



Spotify

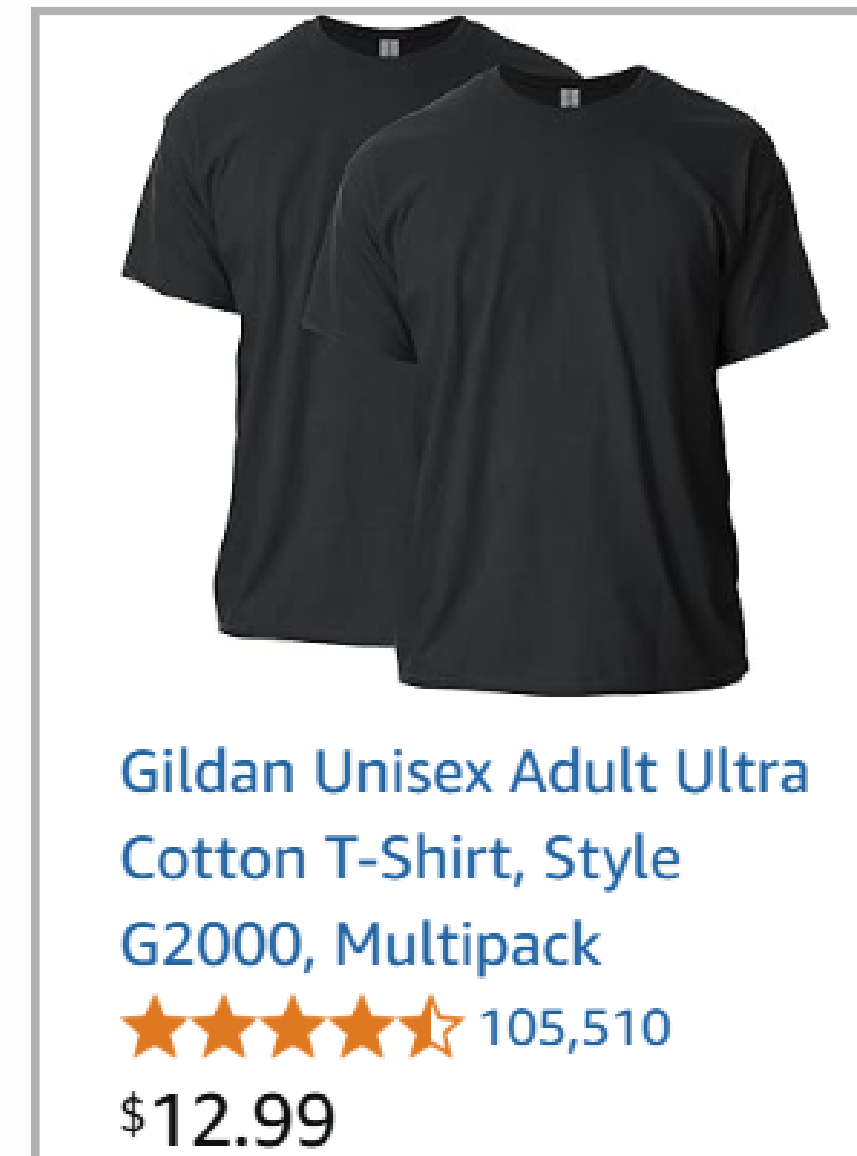
ภาพที่ 6-2 : ตัวอย่างระบบแนะนำ (Recommendation System) (ต่อ)



Shopee



Lazada



Amazon

ภาพที่ 6-2 : ตัวอย่างระบบแนะนำ (Recommendation System) (ต่อ)

• ประเภทของระบบแนะนำ (Types of Recommendation System)

1. Content-Based Filtering

- **หลักการ** : ระบบจะแนะนำสินค้าโดยพิจารณาจาก **ความสนใจของผู้ใช้** (เช่น เสื้อผ้า รองเท้ากีฬา สมาร์ทโฟน) หรือ **พฤติกรรมของผู้ใช้** (เช่น ชื้อ, กดดู, กดไลก์) แล้วแนะนำสินค้าที่คล้ายคลึงกัน
- **ข้อดี** : ไม่ต้องใช้ข้อมูลจากผู้ใช้อื่น (ไม่เกิดปัญหา Cold-start)
- **ข้อเสีย** : ระบบมักจะแนะนำแต่สินค้าที่คล้ายคลึงกับสิ่งที่ผู้ใช้ชอบ (Lack of serendipity)

2. Collaborative Filtering

- **หลักการ** : ระบบจะแนะนำสินค้าที่คุณไม่เคยซื้อมาก่อน แต่คนอื่นที่มีพฤติกรรมซื้อ (หรือกดดู, กดไลก์) คล้ายกับคุณเคยซื้อมาแล้ว
- **ข้อดี** : แนะนำสินค้าใหม่ ๆ ที่ผู้ใช้ไม่เคยเห็นมาก่อน (Serendipity)
- **ข้อเสีย** : มีปัญหา Cold-start สำหรับผู้ใช้ใหม่ รวมถึงปัญหาความเบาบางของข้อมูลพฤติกรรม (Sparsity)

3. Hybrid Approach

- **หลักการ** : ใช้ทั้ง Content-Based และ Collaborative เช่น Netflix ใช้ทั้งคุณสมบัตินของหนังและพฤติกรรมผู้ใช้
- **ข้อดี** : สามารถแก้ปัญหา Cold-start สำหรับผู้ใช้ใหม่ และให้คำแนะนำที่หลากหลาย (Serendipity)
- **ข้อเสีย** : มีความซับซ้อนในการพัฒนาและใช้ทรัพยากรในการประมวลผลสูงกว่า

• เทคนิคในการพัฒนาระบบแนะนำ (Techniques of Recommendation System)

ตารางที่ 6-1 : เปรียบเทียบเทคนิคในการพัฒนาระบบแนะนำ

เทคนิค	หลักการ / แนวคิด	ข้อดี	ข้อเสีย
Euclidean Distance	เป็นคำนวณระยะห่างระหว่างจุด 2 จุด เช่น โปรไฟล์ผู้ใช้ กับ โปรไฟล์ผู้ใช้คนอื่น ๆ	- เข้าใจง่าย - เหมาะกับข้อมูลที่คำนึงถึงความแตกต่างเชิงปริมาณ เช่น การให้ rating	- ไม่เหมาะกับข้อมูลมิติสูง - ข้อมูลที่มีสเกลต่างกันต้อง normalize
Cosine Similarity	เป็นการคำนวณค่าของมุมระหว่างเวกเตอร์ 2 เวกเตอร์ เช่น เวกเตอร์ของโปรไฟล์ผู้ใช้ กับ โปรไฟล์ผู้ใช้คนอื่น ๆ	- เข้าใจง่าย - เหมาะกับข้อมูลมิติสูง เช่น text	- ไม่คำนึงถึงความแตกต่างเชิงปริมาณ เช่น การให้ rating
Singular Value Decomposition (SVD)	เป็นการทำเทคนิค Matrix Factorization เพื่อลดมิติ หรือสกัดปัจจัยแฝง (latent factors) เช่น ความชอบที่ซ่อนอยู่	- ลดปัญหา sparsity ได้ดี - ลดมิติของข้อมูลได้ดี	- เมื่อมี user ใหม่ ต้อง train ใหม่ หรือทำ incremental SVD - ไม่เหมาะกับ real-time update
Term Frequency–Inverse Document Frequency (TF-IDF)	เป็นการใช้หลักการนับความถี่ของคำในโพสต์, ข้อความ, หรือเอกสาร	- เหมาะกับข้อมูลเชิงข้อความ - คำนวณง่ายและมีประสิทธิภาพ - ใช้ร่วมกับ Cosine Similarity ได้ดี	- ไม่เข้าใจความหมายเชิงบริบทของคำ - ไม่ดีนักกับคำพ้องรูป
Deep Learning	เป็นการเรียนรู้ pattern ทั้งจากโปรไฟล์ผู้ใช้, พฤติกรรมผู้ใช้, และ context (เช่น เวลา, สถานที่, อุปกรณ์ที่ใช้)	- จับความสัมพันธ์ซับซ้อนได้ดีมาก - รองรับข้อมูลหลายชนิด (ข้อความ, ภาพ, เสียง, วิดีโอ)	- ต้องการข้อมูลและทรัพยากรสูง - การเทรน/ปรับโมเดลใช้เวลานาน - ยากต่อการตีความ (black box)

6.1.1 Euclidean Distance

สูตรการคำนวณ

$$d = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}$$

เมื่อ

d คือ ระยะทางระหว่างจุด

x_i คือ ค่าในมิติที่ i ของจุด x

y_i คือ ค่าในมิติที่ i ของจุด y

m คือ จำนวนมิติ

ตัวอย่างการคำนวณ

จงหาระยะทางระหว่างจุด (1, 2) และจุด (4, 6) ด้วย Euclidean Distance

วิธีทำ

$$d = \sqrt{(1 - 4)^2 + (2 - 6)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

6.1.2 Cosine Similarity

สูตรการคำนวณ

$$\cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

เมื่อ

$\cos(\theta)$ คือ มุมระหว่างเวกเตอร์ $\mathbf{x}$ และ $\mathbf{y}$

$\mathbf{x} \cdot \mathbf{y}$ คือ Dot product ระหว่างเวกเตอร์ $\mathbf{x}$ และ $\mathbf{y}$

$\|\mathbf{x}\|$ คือ ขนาด ของเวกเตอร์ $\mathbf{x}$

$\|\mathbf{y}\|$ คือ ขนาด ของเวกเตอร์ $\mathbf{y}$

ตัวอย่างการคำนวณ

จงหาระยะทางระหว่างจุด (1, 2) และจุด (4, 6) ด้วย Cosine Distance

วิธีทำ

$$\mathbf{x} \cdot \mathbf{y} = (1 \times 4) + (2 \times 6) = 16$$

$$\|x\| = \sqrt{1^2 + 2^2} = \sqrt{5}, \quad \|y\| = \sqrt{4^2 + 6^2} = \sqrt{52}$$

$$\cos(\theta) = \frac{16}{\sqrt{5}\sqrt{52}} = 0.992$$

Cosine Similarity	Cosine Distance	มุม	ความหมาย
1	0	0°	เหมือนกันมาก
0	1	90°	ไม่สัมพันธ์
-1	2	180°	ตรงข้าม

*Cosine Distance = 1 - Cosine Similarity

6.1.3 TF-IDF

แนวคิด

- พิจารณา คำซ้ำทั้งใน Document เดียวกัน และต่าง Document กัน
- ค่า TF-IDF จะสูง ถ้าใน Document เดียวกัน ซ้ำเยอะ แต่ต่าง Document กัน ซำน้อย

สูตรการคำนวณ

- คำนวณค่า TF

$$TF(t, d) = \frac{\text{จำนวนครั้งที่คำ } t \text{ ปรากฏใน document } d}{\text{จำนวนคำทั้งหมดใน document } d}$$

- คำนวณค่า IDF

$$IDF(t) = \log \frac{N}{n(t)} \quad \text{บางทฤษฎีใช้สมการ} \quad IDF(t) = \ln \left(\frac{N + 1}{n(t) + 1} \right) + 1$$

เมื่อ

N = จำนวน document ทั้งหมด

$n(t)$ = จำนวน document ที่มีคำ t

- คำนวณค่า TF-IDF

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

ตัวอย่างการคำนวณ

จงแสดงการทำ Text Vectorization ด้วยวิธี Bag of Words เกี่ยวกับข้อมูล รีวิวภาพยนตร์ ซึ่งมีรายการ ดังนี้

- D1: หนังดีสนุกมาก นักแสดงดี
- D2: หนังน่าเบื่อ เนื้อเรื่องซ้ำ
- D3: นักแสดงดี แต่เนื้อเรื่องธรรมดา ธรรมดา
- D4: หนังสนุก เนื้อเรื่องดี

วิธีทำ

ทำการตัดคำ (Word Tokenization) และสร้างรายการคำศัพท์ (Vocabulary) ทั้งหมด 10 คำ ดังนี้
[หนัง, สนุก, มาก, นักแสดง, ดี, น่าเบื่อ, เนื้อเรื่อง, ซ้ำ, แต่, ธรรมดา]

จากนั้น นำไปทำ Text Vectorization ได้

Document	ซ้ำ	ดี	ธรรมดา	นักแสดง	น่าเบื่อ	มาก	สนุก	หนัง	เนื้อเรื่อง	แต่	จำนวนคำทั้งหมดใน document
D1	0	2	0	1	0	1	1	1	0	0	6
D2	1	0	0	0	1	0	0	1	1	0	4
D3	0	1	2	1	0	0	0	0	1	1	6
D4	0	1	0	0	0	0	1	1	1	0	4
$n(t)$	1	3	1	2	1	1	2	3	3	1	

- คำนวณค่า TF

$$TF(t, d) = \frac{\text{จำนวนครั้งที่คำ } t \text{ ปรากฏใน document } d}{\text{จำนวนคำทั้งหมดใน document } d}$$

Document	ข้า	ดี	รรรมดา	นักแสดง	น่าเบื่อ	มาก	สนุก	หนัง	เนื้อเรื่อง	แต่
D1	0/6 = 0	2/6 = 0.33	0/6 = 0	1/6 = 0.17	0/6 = 0	1/6 = 0.17	1/6 = 0.17	1/6 = 0.17	0/6 = 0	0/6 = 0
D2	1/4 = 0.25	0/4 = 0	0/4 = 0	0/4 = 0	1/4 = 0.25	0/4 = 0	0/4 = 0	1/4 = 0.25	1/4 = 0.25	0/4 = 0
D3	0/6 = 0	1/6 = 0.17	2/6 = 0.33	1/6 = 0.17	0/6 = 0	0/6 = 0	0/6 = 0	0/6 = 0	1/6 = 0.17	1/6 = 0.17
D4	0/4 = 0	1/4 = 0.25	0/4 = 0	0/4 = 0	0/4 = 0	0/4 = 0	1/4 = 0.25	1/4 = 0.25	1/4 = 0.25	0/4 = 0

- คำนวณค่า IDF

$$IDF(t) = \log \frac{N}{n(t)}, N = 4$$

	ข้า	ดี	รรรมดา	นักแสดง	น่าเบื่อ	มาก	สนุก	หนัง	เนื้อเรื่อง	แต่
$n(t)$	1	3	1	2	1	1	2	3	3	1
$IDF(t)$	$\log(4/1)=0.6$	$\log(4/3)=0.12$	$\log(4/1)=0.6$	$\log(4/2)=0.3$	$\log(4/1)=0.6$	$\log(4/1)=0.6$	$\log(4/2)=0.3$	$\log(4/3)=0.12$	$\log(4/3)=0.12$	$\log(4/1)=0.6$

- คำนวณค่า TF-IDF

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

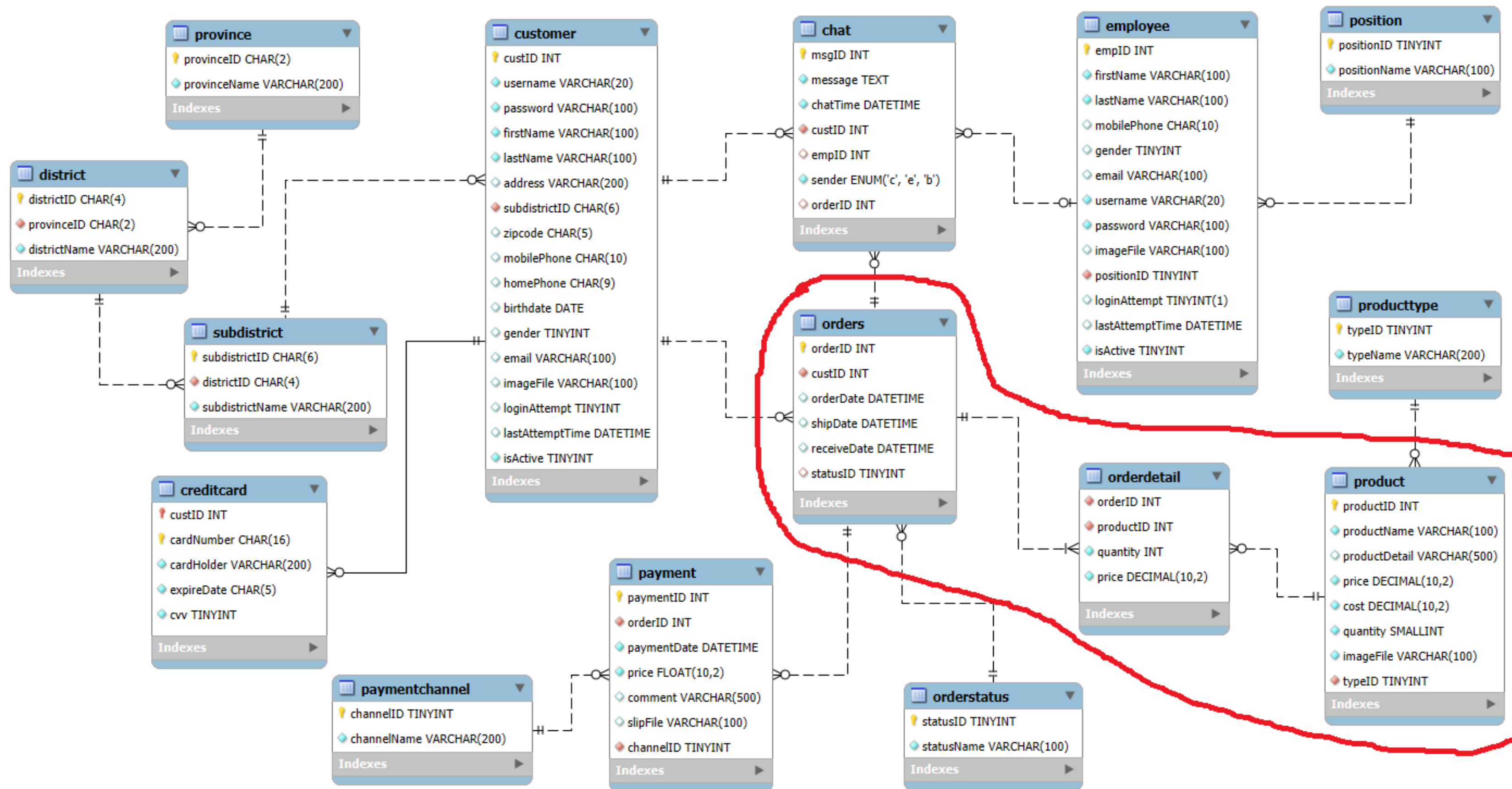
Document	ช้า	ดี	ธรรมดา	นักแสดง	น่าเบื่อ	มาก	สนุก	หนัง	เนื้อเรื่อง	แต่
D1	0*0.6= 0	0.33*0.12= 0.04	0*0.6= 0	0.17*0.3= 0.05	0*0.6= 0	0.17*0.6= 0.1	0.17*0.3= 0.05	0.17*0.12= 0.02	0*0.12= 0	0*0.6= 0
D2	0.25*0.6= 0.15	0*0.12= 0	0*0.6= 0	0*0.3= 0	0.25*0.6= 0.15	0*0.6= 0	0*0.3= 0	0.25*0.12= 0.03	0.25*0.12= 0.03	0*0.6= 0
D3	0*0.6= 0	0.17*0.12= 0.02	0.33*0.6= 0.2	0.17*0.3= 0.05	0*0.6= 0	0*0.6= 0	0*0.3= 0	0*0.12= 0	0.17*0.12= 0.02	0.17*0.6= 0.1
D4	0*0.6= 0	0.25*0.12= 0.03	0*0.6= 0	0*0.3= 0	0*0.6= 0	0*0.6= 0	0.25*0.3= 0.08	0.25*0.12= 0.03	0.25*0.12= 0.03	0*0.6= 0

6.2 การพัฒนา ML/AI Model สำหรับแนะนำสินค้า

- ใช้เทคนิค Euclidean Distance และ ไลบรารี Scikit-Learn ในการสร้าง ML/AI Model สำหรับแนะนำสินค้า
- ใช้ข้อมูลในระบบซื้อขายออนไลน์เพื่อ Train/Test Model
- วัดประสิทธิภาพ Model ด้วยค่า Precision/Recall/F1-Score

ตารางที่ 6-2 : ตัวอย่างข้อมูลที่ใช้ในการพัฒนา Model สำหรับแนะนำสินค้า

	custID	เสื้อแขนยาว	เสื้อแขนสั้น	กางเกงขายาว	กางเกงขาสั้น	หมวกแก๊ป	หมวกไหมพรม	รองเท้าผ้าใบ	รองเท้าแตะ	ถุงเท้ายาว	ถุงเท้าสั้น
▶	1	2	0	6	0	3	2	1	0	0	0
	2	0	0	5	0	4	0	0	0	0	0
	3	3	0	3	5	2	0	0	0	0	0
	4	1	1	0	0	0	1	0	0	0	0
	5	1	0	0	0	0	2	0	0	0	0
	6	0	0	0	0	0	0	0	0	0	0
	7	0	0	0	0	0	0	0	0	0	0
	8	0	0	0	0	0	0	0	0	0	0



ภาพที่ 6-3 : ER-Diagram

```

1  CREATE ALGORITHM=UNDEFINED
2  DEFINER=`root`@`localhost`
3  SQL SECURITY DEFINER
4  VIEW `recommend_view` AS
5  √ SELECT
6      c.custID AS custID,
7      SUM(CASE WHEN p.productName = 'เส้นแขนยาว' THEN od.quantity ELSE 0 END) AS เส้นแขนยาว,
8      SUM(CASE WHEN p.productName = 'เส้นแขนสั้น' THEN od.quantity ELSE 0 END) AS เส้นแขนสั้น,
9      SUM(CASE WHEN p.productName = 'กางเกงขายาว' THEN od.quantity ELSE 0 END) AS กางเกงขายาว,
10     SUM(CASE WHEN p.productName = 'กางเกงขาสั้น' THEN od.quantity ELSE 0 END) AS กางเกงขาสั้น,
11     SUM(CASE WHEN p.productName = 'หมวกแก๊ป' THEN od.quantity ELSE 0 END) AS หมวกแก๊ป,
12     SUM(CASE WHEN p.productName = 'หมวกไหมพรม' THEN od.quantity ELSE 0 END) AS หมวกไหมพรม,
13     SUM(CASE WHEN p.productName = 'รองเท้าผ้าใบ' THEN od.quantity ELSE 0 END) AS รองเท้าผ้าใบ,
14     SUM(CASE WHEN p.productName = 'รองเท้าแตะ' THEN od.quantity ELSE 0 END) AS รองเท้าแตะ,
15     SUM(CASE WHEN p.productName = 'ถุงเท้ายาว' THEN od.quantity ELSE 0 END) AS ถุงเท้ายาว,
16     SUM(CASE WHEN p.productName = 'ถุงเท้าสั้น' THEN od.quantity ELSE 0 END) AS ถุงเท้าสั้น
17  √ FROM
18      (((customer c
19  √ LEFT JOIN
20      orders o ON c.custID = o.custID)
21  √ LEFT JOIN
22      orderdetail od ON o.orderID = od.orderID)
23  √ LEFT JOIN
24      product p ON od.productID = p.productID)
25  √ GROUP BY
26      c.custID;

```

ภาพที่ 6-4 : View (SQL)

```

1 CREATE DEFINER=`root`@`localhost` PROCEDURE `recommend_procedure`()
2 BEGIN
3     SET SESSION group_concat_max_len = 1000000;
4
5     SET @DynamicSQL = '';
6
7     SELECT GROUP_CONCAT(
8         'SUM(CASE WHEN p.productName = '', productName, '' THEN od.quantity ELSE 0 END) AS `', productName, ``
9         SEPARATOR ', '
10    )
11    INTO @DynamicSQL
12    FROM (
13        SELECT DISTINCT productName FROM product
14    ) AS product_names;
15
16    SET @DynamicSQL = CONCAT('
17    SELECT
18        c.custID,
19        ', @DynamicSQL, '
20    FROM
21        customer c
22    LEFT JOIN
23        orders o ON c.custID = o.custID
24    LEFT JOIN
25        orderdetail od ON o.orderID = od.orderID
26    LEFT JOIN
27        product p ON od.productID = p.productID
28    GROUP BY
29        c.custID
30    ');
31
32    PREPARE stmt FROM @DynamicSQL;
33    EXECUTE stmt;
34    DEALLOCATE PREPARE stmt;
35 END

```

ภาพที่ 6-5 : Store Procedure (SQL)

- เขียนโค้ดเพื่อสร้าง ML/AI model

```
1  # Import Python Library
2  from sklearn.neighbors import NearestNeighbors
3  import pandas as pd
4
5  class RecommendationModel:
6      # Build (กำหนดโครงสร้างของโมเดล)
7      def __init__(self, n_neighbors=1, metric='minkowski', p=2):
8          self.net = NearestNeighbors(n_neighbors=n_neighbors, metric=metric, p=p)
9
10     # Train (ฝึกโมเดลด้วยข้อมูลลูกค้าคนอื่น)
11     def train(self, x_other_users: pd.DataFrame):
12         self.net.fit(x_other_users)
13
14     # Test (หาสินค้าที่มีพฤติกรรมคล้ายกับ login user)
15     def find_similar_user(self, x_login_user: pd.DataFrame):
16         index = self.net.kneighbors(x_login_user)[1][0][0]
17         return index
18
19     # Get product names from the DataFrame
20     def get_product_name(self, x_login_user: pd.DataFrame, x_similar: pd.DataFrame) -> str:
21         product_count = len(x_login_user.columns)
22         product_names = []
23
24         for i in range(product_count):
25             if (x_login_user.iloc[0, i] == 0 and x_similar.iloc[0, i] > 0):
26                 product_names.append(str(x_similar.columns[i]))
27
28         product_names_str = ', '.join(f'"{name}"' for name in product_names)
29         return product_names_str
```

```

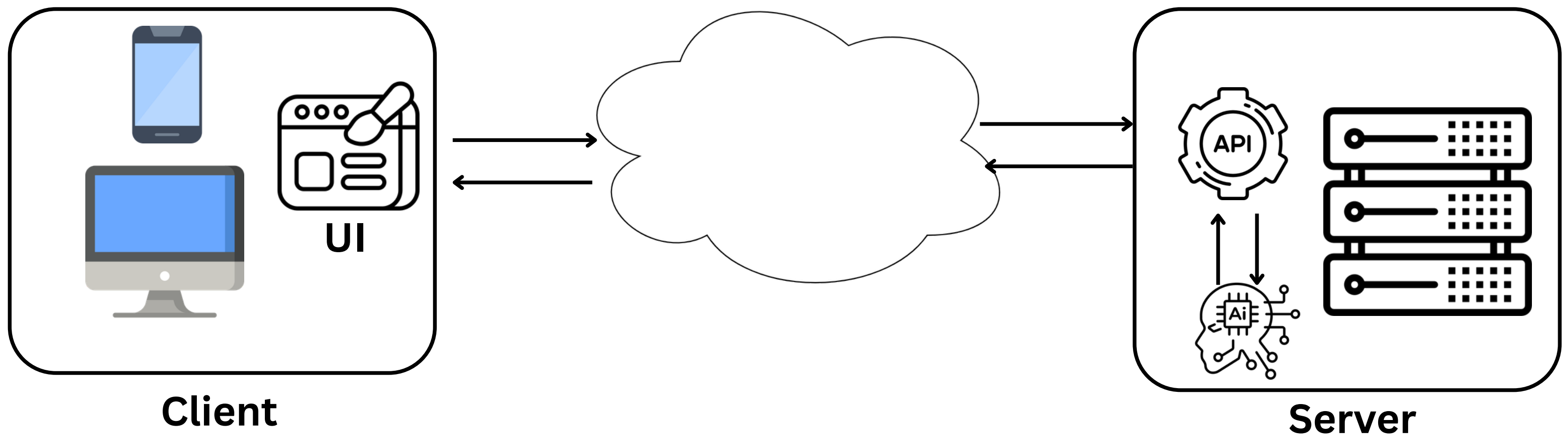
30
31 # For testing purposes
32 if __name__ == "__main__":
33     # Example training data
34     # Suppose we have 4 customers, each row = customer
35     # Columns = a number of purchased products
36     train_data = [
37         [1, 1, 2], # Cust1
38         [0, 1, 5], # Cust2
39         [3, 1, 0], # Cust3
40         [0, 4, 0]  # Cust4
41     ]
42
43     # สร้าง DataFrame พร้อม column และ index
44     train_df = pd.DataFrame(train_data,
45                             columns=["ProductA", "ProductB", "ProductC"],
46                             index=["Cust1", "Cust2", "Cust3", "Cust4"]
47     )
48     print("Training Data:")
49     print(train_df)
50
51     # Example login user who bought A=1, B=0, C=2 (similar to Cust1)
52     test_data = [[1, 0, 2]]
53     login_user = pd.DataFrame(test_data, columns=["ProductA", "ProductB", "ProductC"])
54     print("\nTest Data (User Login):")
55     print(login_user)

```

```
56
57     # Build and train model
58     model = RecommendationModel()
59     model.train(train_df)
60
61     # Test with a Login User
62     similar_index = model.find_similar_user(login_user)
63     similar_customer_id = train_df.index[similar_index]
64     similar_customer_data = train_df.loc[[similar_customer_id]]
65     print("\nMost similar customer data:\n", similar_customer_data)
66
67     # Show recommended products
68     print("\nMost similar products:")
69     print(model.get_product_name(login_user, similar_customer_data))
```

6.3 การพัฒนา API สำหรับแนะนำสินค้า

- **API (Application Programming Interface)** คือ ตัวกลางในการสื่อสารข้อมูลระหว่างระบบคอมพิวเตอร์ เช่น การรับส่งข้อมูลระหว่างเครื่องคอมพิวเตอร์ของผู้ใช้งาน (Client) กับเครื่องเซิร์ฟเวอร์ (Server)
- ใช้ flask ในการพัฒนา API



ภาพที่ 6-6 : สถาปัตยกรรม Client-Server

ติดตั้ง Library ที่จำเป็น : pip install flask flask-cors mysql-connector-python

```
1  # -----
2  # ♦ Import Python libraries
3  # -----
4  from flask import Flask           # สำหรับสร้าง web API
5  from flask import jsonify         # สำหรับข้อมูล JSON
6  from flask_cors import CORS       # เพื่อเปิด CORS ให้สามารถเรียกจาก front-end ได้
7  import mysql.connector as mysql   # สำหรับเชื่อมต่อ MySQL
8  import pandas as pd               # สำหรับอ่านข้อมูลจาก MySQL
9  import warnings                   # สำหรับปิด warning จาก python
10 from model import RecommendationModel # สำหรับ import ml model
11
12 # -----
13 # ♦ Configuration
14 # -----
15 warnings.filterwarnings("ignore") # ปิด warning
16 app = Flask(__name__) # สร้าง Flask app เพื่อใช้สร้าง API
17 CORS(app) # เปิดใช้งาน CORS เพื่อให้ front-end เรียก API ได้
18
```

```
19 # -----
20 # ♦ Create a MySQL connection
21 # -----
22 def get_db_connection():
23     try:
24         conn = mysql.connect(
25             host="localhost",
26             database="shopdee",
27             user="root",
28             password="1234"
29         )
30         if conn.is_connected():
31             return conn
32         else:
33             raise Exception("Failed to connect to MySQL")
34     except mysql.Error as err:
35         return jsonify({
36             "status": False,
37             "message": f"MySQL connection error: {err}"
38         }), 500
39
40
```

```

41 # -----
42 # ♦ Recommendation API
43 # -----
44 @app.route('/api/recommend/<int:id>', methods=['GET'])
45 def recommend(id):
46     conn = get_db_connection() # Establish connection
47     try:
48         # Load data
49         sql = "call recommend_procedure()"
50         x = pd.read_sql(sql, conn, index_col=['custID'])
51
52         # Define prediction data and training data
53         x_login_user = x[x.index == id] # prediction data
54         x_other_users = x.drop(id) # training data
55
56         # Filter only columns where login user has value > 0
57         cols = x_login_user.columns[x_login_user.iloc[0] > 0]
58         x_login_user_filtered = x_login_user[cols]
59         x_other_users_filtered = x_other_users[cols]
60
61         # Build and train model
62         model = RecommendationModel()
63         model.train(x_other_users_filtered)
64

```

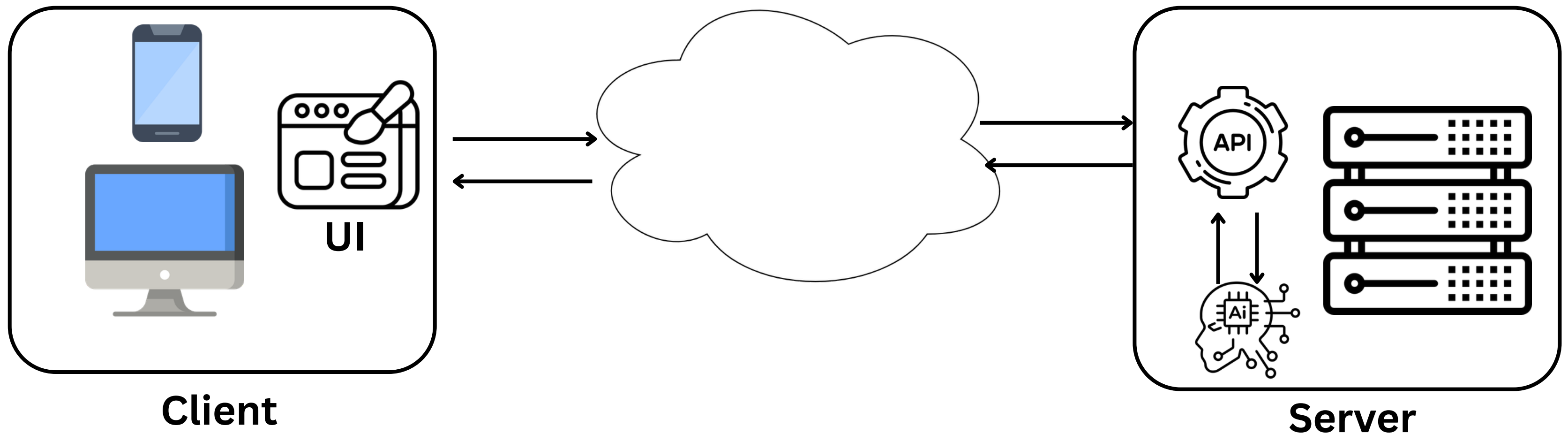
```

65     # Predict (หาลูกค้าคนอื่นที่มีพฤติกรรมคล้ายมากที่สุด)
66     index = model.find_similar_user(x_login_user_filtered) #user index
67     x_similar = x_other_users.iloc[[index]] #user data
68
69     # หาชื่อสินค้าที่จะแนะนำ
70     product_names = model.get_product_name(x_login_user, x_similar)
71
72     if len(product_names) == 0:
73         return jsonify([]), 200
74     else:
75         conn = get_db_connection()
76
77     # แสดงรายการสินค้าที่ต้องการแนะนำ
78     sql = f'SELECT * FROM product WHERE productName IN ({product_names})'
79     products = pd.read_sql(sql, conn)
80
81     # Return the result
82     return jsonify(products.to_dict(orient='records')), 200
83
84 except Exception as e:
85     return jsonify({"status": False, "message": str(e)}), 500
86 finally:
87     conn.close() # ปิดการเชื่อมต่อ กับ MySQL
88
89 # -----
90 # ♦ Run API server
91 # -----
92 if __name__ == "__main__":
93     app.run(host="0.0.0.0", port=8000, debug=True)

```

6.4 การพัฒนา Web App สำหรับแนะนำสินค้า

- ใช้ TypeScript + Next.js + Tailwind CSS พัฒนาในส่วนของ UI (Front-End)
- ทำการเชื่อมต่อไปยัง API เพื่อเรียกใช้งาน ML/AI Model



ภาพที่ 6-7 : สถาปัตยกรรม Client-Server

ขั้นตอนการพัฒนา Web App สำหรับแนะนำสินค้า

1. เริ่มสร้างโปรเจกต์

`npx create-next-app@latest my-app --typescript`

แล้วเลือกตอบ “Yes” หรือ “No” ดังนี้

✓ Would you like to use ESLint? ... No / Yes

✓ Would you like to use Tailwind CSS? ... No / Yes

✓ Would you like your code inside a `src/` directory? ... No / Yes

✓ Would you like to use App Router? (recommended) ... No / Yes

✓ Would you like to use Turbopack for `next dev`? ... No / Yes

✓ Would you like to customize the import alias (`@/\*` by default)? ... No / Yes

2. ทำการติดตั้ง Axios เพื่อให้สามารถเรียกใช้ API ได้ (หากมีการตั้งชื่อโปรเจกต์ให้ใช้คำสั่ง `cd my-app`)

`npm install axios`

3. รัน Web App

`npm run dev`

4. ทดสอบเรียก url จาก web browser

`http://localhost:3000`

ระบุเครื่องหมายจุด . แทนชื่อโปรเจกต์ หากไม่ต้องการให้สร้าง folder ใหม่

5. จากนั้นแก้ไขโค้ดในไฟล์ /src/app/page.tsx

ตัวอย่างการ prompt เพื่อให้ AP ทำการสร้างหน้าแสดงรายการสินค้าแนะนำ

ขอ source code ที่ใช้ TypeScript + Tailwind CSS เพื่อพัฒนา Modern Web App. ในการแสดงรายการสินค้าแนะนำที่ได้จาก AP โดยมีรายละเอียดดังนี้

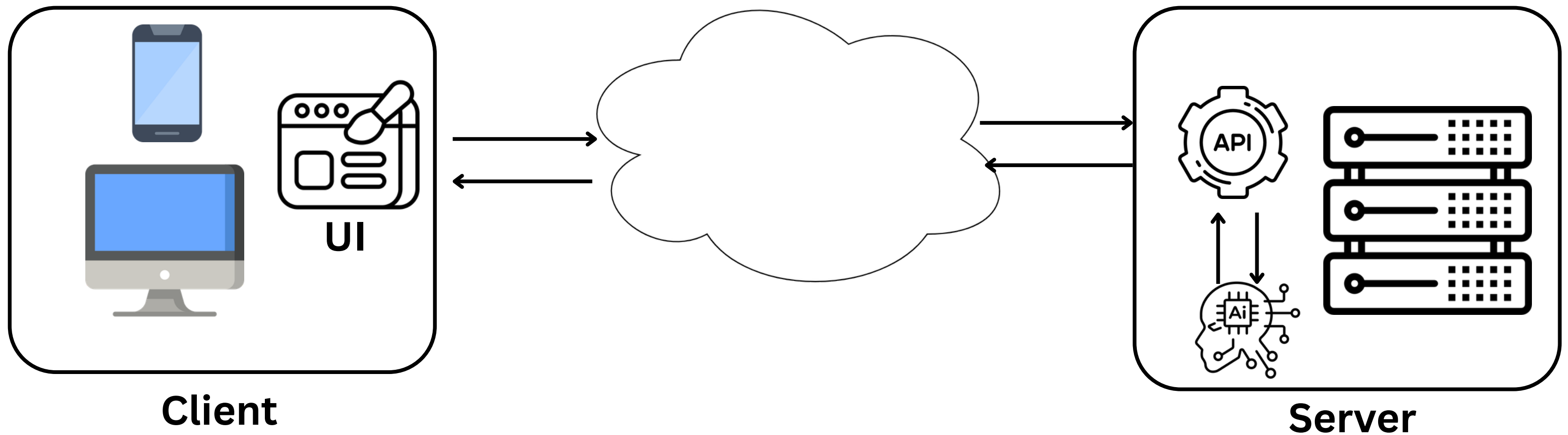
API Endpoint : `http://127.0.0.1:8000/api/recommend/2`

API Output:

```
[
  {
    "cost": 200.0,
    "imageFile": "img1.jpg",
    "price": 250.0,
    "productDetail": "สีขาว ผ้ายืด ขนาด M",
    "productID": 1,
    "productName": "เสื้อแขนยาว",
    "quantity": 10,
    "typeID": 1
  }
]
```

6.5 การพัฒนา Mobile App สำหรับแนะนำสินค้า

- ใช้ Dart + Flutter พัฒนาในส่วนของ UI (Front-End)
- ทำการเชื่อมต่อไปยัง API เพื่อเรียกใช้งาน ML/AI Model



ภาพที่ 6-8 : สถาปัตยกรรม Client-Server

ขั้นตอนการพัฒนา Mobile App สำหรับแนะนำสินค้า

1. เริ่มสร้างโปรเจกต์

flutter create **hello_app**

← ระบุเครื่องหมายจุด . แทนชื่อโปรเจกต์ หากไม่ต้องการให้สร้าง folder ใหม่

2. รัน Mobile App (หากมีการตั้งชื่อโปรเจกต์ให้ใช้คำสั่ง cd **hello-app**)

flutter run

หมายเหตุ : - หากเชื่อมต่อมือถือหรือเปิด Emulator ระบบจะทำการเลือก device ให้อัตโนมัติ
- หากไม่ได้เชื่อมต่ออะไร ระบบจะให้เลือกว่าจะรัน win app. หรือ web app.

3. ดูผลลัพธ์จาก Web browser/Emulator/Device

4. จากนั้นแก้ไขโค้ดในไฟล์ /lib/main.dart

ตัวอย่างการ prompt เพื่อให้ API ทำการสร้างหน้าแสดงรายการสินค้าแนะนำ

ขอ source code สำหรับไฟล์ main.dart ที่ใช้ Dart + Flutter เพื่อพัฒนา Modern Mobile App.
ในการแสดงรายการสินค้าแนะนำ ที่ได้จาก AP โดยมีรายละเอียดดังนี้

API Endpoint : `http://127.0.0.1:8000/api/recommend/2`

API Output:

```
[
  {
    "cost": 200.0,
    "imageFile": "img1.jpg",
    "price": 250.0,
    "productDetail": "สีขาว ผ้ายืด ขนาด M",
    "productID": 1,
    "productName": "เสื้อแขนยาว",
    "quantity": 10,
    "typeID": 1
  }
]
```



THANK → YOU!

Get In Touch With Us!

Phone

0-2692-2360-4

Facebook

<https://www.facebook.com/ComSci.CPC/>

Address

58 ถนนวิภาวดีรังสิต แขวง
รัชดาภิเษก เขตดินแดง

